PG-MDP: Profile-Guided Memory Dependence Prediction for Area-Constrained Cores

MICRO 2026 Submission **#1358** – Confidential Draft – Do NOT Distribute!!

Abstract

Memory Dependence Prediction (MDP) is a speculative technique to predict which stores, if any, a given load will depend on. Area-constrained cores are increasingly relevant in various applications such as energy-efficient or edge systems, and often have limited space for MDP tables. This leads to a high rate of false dependencies as memory independent loads alias with unrelated predictor entries, causing unnecessary stalls in the processor pipeline.

The conventional way to address this problem is with greater predictor size or complexity, but this is unattractive on area-constrained cores. This paper demonstrates that targeting the **predictor working set** delivers the majority of available performance without scaling any hardware structures. We achieve this with profile-guided memory dependence prediction (PG-MDP), a software co-design to label consistently memory independent loads via their opcode and remove them from the MDP working set. These loads bypass querying the MDP on dispatch and always issue as soon as possible. In the event that a labelled load incorrectly passes a store to the same address, a rollback is triggered as usual but no new MDP entry is created. Across the SPECspeed 2017 suites, PG-MDP reduces MDP queries by 73%, false dependencies by 76%, and improves geomean IPC for a small (ROB=128) simulated core by 3.6% (to within 1.5% of the IPC when using 16x more predictor entries), with **no area cost** and no additional instruction bandwidth.

1 Introduction

Memory dependence prediction (MDP) is a speculative technique used in out-of-order processors to increase available instruction-level parallelism (ILP) by predicting the stores on which a given load instruction will depend, as well as identifying loads which are memory independent.

Recent work [**?**] exploring MDP designs in modern commercial processors finds that the storage budget allocated to memory dependence predictors is extremely small, with results suggesting even high-performance cores may not break 1KB of storage, and area-constrained efficiency-focused cores could be using as little as 50-100 bytes. This is unsurprising when considering the design constraints of these predictors; because MDPs are typically queried by both loads and stores at dispatch, their tables must be highly multi-ported to ensure dispatch is never blocked by memory operations waiting to receive predictions. For example, the open-source XiangShan processor [**?**] implements a Store Sets-based [7] predictor with 8 read ports. This places a high premium on area allocated to memory dependence predictors, which is further exacerbated in area-constrained cores where additional area is highly contested by performance-critical components (e.g. branch prediction), and low power usage is a top priority. As such, these smaller cores employ smaller predictors and struggle with a high rate of false dependencies due to hash collisions/aliasing, which unnecessarily stalls loads on non-dependent stores.

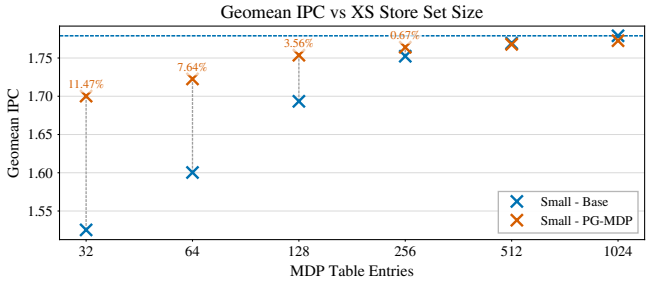

Figure 1: Base and improved IPC for increasing XS Store Set [23] sizes on a small core (ROB=128). With PG-MDP, 128 entries deliver within 1.5% of available IPC from 1024 entries. PG-MDP plots past 256 entries are omitted as performance is near-identical.

The importance of such area-constrained cores has also grown over time. Modern systems increasingly deploy heterogeneous processors containing both performance and efficiency-focused core designs, combining larger high-performance cores with small energy-efficient (but still out-of-order) cores, as seen in ARM's big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. These efficiency cores share die area with high-performance cores under strict budgets for both area and power. This pattern is no longer confined to mobile devices, and this heterogeneous design can now be found across mobile, desktop, and server designs. Even as processors in high-end mobile devices scale to rival those found in laptops, they are often paired with efficiency cores to maximize battery life. Area-constrained cores are further found as standalone cores in edge systems, or efficiency tiles in chiplet architectures. In all these cases, out-of-order execution components are scaled to fit the given area envelope rather than to maximize performance. As these area-constrained cores are increasingly tasked with demanding general-purpose computation while striving to consume as little energy as possible, increasing ILP without incurring additional area or power costs becomes more pressing.

False dependencies in MDP can be addressed by increasing predictor capacity, but as established this is either unattractive or simply infeasible. Instead, this paper improves accuracy by reducing the **predictor working set**. We define the predictor working set as the set of instructions that actively index into the predictor during a given execution window. A typical MDP working set is every load and store instruction, as seen in Store Sets [7] and many of the predictors listed in [**?**]. There exist several sophisticated hardware techniques which can at least partially reduce the MDP working set, such as load elimination [4], memory renaming [15],

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX

or value prediction [?], but each of these also requires space for large, power-hungry predictors, as well as abundant ILP to return the most benefit, which are precisely the things an area-constrained core lacks. Furthermore, prior work demonstrates that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight dependencies even across varying inputs [4, 9, 16], and so these loads are ideal candidates to be removed from the MDP working set via hardware-software co-design in the ISA without incurring area or power cost beyond trivial additional decode logic.

We achieve this with **profile-guided memory dependence prediction (PG-MDP)**, a software co-design which identifies consistently memory-independent loads via profiling and labels them via an alternate opcode, allowing them to bypass MDP queries entirely and always schedule as early as possible. Using a modern variant of the Store Set predictor [7] found in the XiangShan core [?], we show that for an appropriate working set even a small predictor is able to deliver performance competitive with a very large predictor. PG-MDP reduces average runtime MDP queries by 73% and false dependencies by 76% across SPECspeed 2017, providing a 3.6% geomean IPC gain on a small (ROB = 128) simulated core, and within 1.5% IPC of using an MDP 16x larger, as shown in Figure 1.

Because loads are removed from the working set via their instruction encoding, PG-MDP compares favourably even against cheap and targeted hardware alternatives. A Bloom filter of dependent load PCs, for instance, would still be queried by all loads, and so inherits the same multi-porting constraint currently found in the MDP. It would then also suffer from saturation at any reasonable hardware budget, and would have to be periodically cleared and re-trained to adapt to program phase changes, limiting the accuracy it can achieve while still incurring additional hardware.

1.1 Contributions

This paper makes the following contributions:

- **Reframes MDP Aliasing:** We show that the dominant source of false dependencies in memory dependence prediction does not stem solely from insufficient predictor capacity, but from equally an unnecessarily large predictor working set that captures both memory dependent and independent loads. We demonstrate that framing MDP aliasing as only a capacity problem is misleading, and that shrinking the working set is as effective as growing the predictor.
- **Introduces PG-MDP:** By leveraging profile-guided memory dependence prediction, we reduce dynamic MDP queries by 73% on average across SPECspeed 2017, and false dependencies by 76%.
- **Delivers Zero-Storage IPC Gains:** Using Gem5 [8] to simulate a core of comparable size to efficiency-focused processors found in production today, we demonstrate a 3.6% IPC gain when using a generously-sized (300B) XiangShan-variant Store Sets predictor (referred to as XS Store Sets) [23], without incurring any additional area or power costs.

2 Background

This section outlines the fundamental concepts in scheduling memory operations in out-of-order execution. It then introduces Store Sets as a foundational memory dependence predictor algorithm as well as the modern variation found in the XiangShan processor.

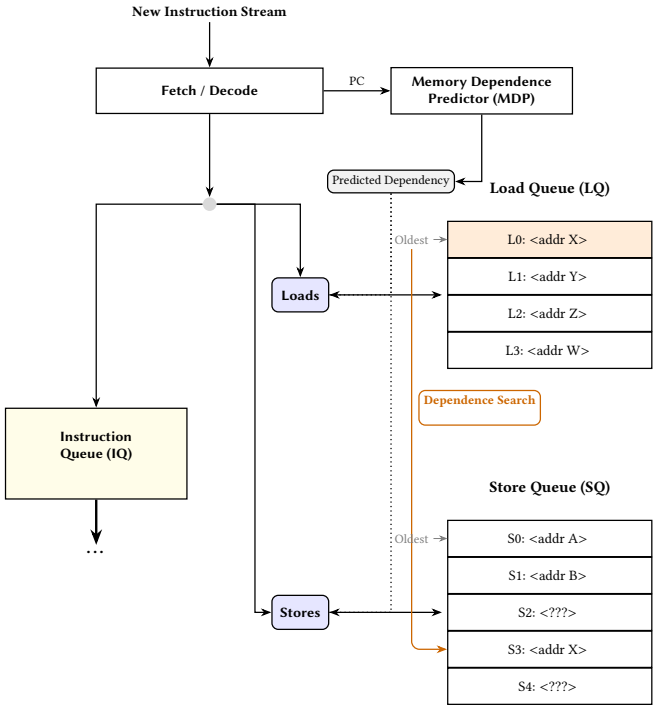

Figure 2: Diagram of components used to schedule memory operations in out-of-order cores. Memory operations are inserted into the IQ according to register dependencies and any predicted memory dependencies, and inserted into the LSQ by program order. MB: We need to be careful about terminology. A Micro reviewer criticised our use the term "dispatch" instead of "issue". ALSO: IQ used before defined. When loads execute they search the SQ for forwarding opportunities. When stores execute they search the LQ for memory order violations.

2.1 Loads in Out-of-Order Execution

A key structure in out-of-order execution is the load-store queue (LSQ), used to hold in-flight memory operations and perform memory disambiguation. When load instructions dispatch, they are inserted into the load queue (LQ), and query the memory dependence predictor (MDP) for a predicted store(s) to wait on before execution. Once the source operands are ready and all predicted dependent stores have executed, loads search backwards through the store queue (SQ) for store-forwarding opportunities. If no matching addresses are found, loads obtain their data from the cache.

When a load searches the SQ, not all earlier stores may have resolved their addresses. As the majority of the time a store's address will not match with the load's, it is often profitable to speculatively ignore these stores and issue the load as soon as possible. When the unresolved store eventually receives its address, it searches forward through the LQ to find loads with matching addresses which have already executed. If an overlap is found, the load has read stale

data from the cache and a memory order violation has occurred. The processor must flush all in-flight instructions from the load onwards and re-fetch.

The goal of the MDP is to minimize violation events while maximizing available ILP, allowing memory independent loads to issue as soon as possible and dependent loads to wait only for the necessary stores. When a load is incorrectly stalled on a store that does not resolve to the same address this is called a false dependency. Simple MDP algorithms typically prioritise reducing the rate of violations over the rate of false dependencies.

2.2 Store Sets

A seminal paper in memory dependence prediction is 'Memory Dependence Prediction using Store Sets' [7]. This is the MDP algorithm implemented in the open source Gem5 simulator [8]. Store Sets works by using Store Set IDs (SSID) to track dependent load-store pairs, assigning each instruction the same ID value in the PC-indexed Store Set ID Table (SSIT). The SSIDs are then used to index the Last-Fetched Store Table (LFST), which holds the sequence number of the most recently issued store with an SSID mapping to that LFST entry. Newly issued loads then access the LFST via their SSID to find the store they're predicted to be dependent on. If a load PC does not map to a valid SSID entry, it is predicted to be memory independent. To forget old dependencies and reduce table pressure, the SSIT and LFST are cyclically cleared after a certain number of issued memory operations.

Store Sets is extremely area efficient, delivering a large portion of available performance with a tiny hardware budget. Its small size and simplicity make it well suited to area-constrained processors. However, especially at smaller sizes, Store Sets struggles with false dependencies due to hash collisions. When a memory independent load hashes to an existing SSID entry, it is incorrectly made dependent on the corresponding store for that entry. The clear period can be made shorter to offset this, but this comes at the trade-off of a higher rate of memory order violations as the predictor must re-learn true dependencies more often. Furthermore, many loads exhibit non-consistent memory dependencies. A load in a loop may only be dependent on a store every other iteration, and as Store Sets has no way to disambiguate these cases it conservatively predicts the load as dependent in every iteration.

2.3 XiangShan Store Sets

XiangShan is an open-source high-performance RISCV processor RTL implementation [24]. XiangShan represents the cutting edge in open-source superscalar processor design, and comes with a Gem5 fork modified to closer model the RTL implementation [23]. This Gem5 fork implements a variation of the Store Sets predictor that offers a closer representation to implementations found in real processors. From here on this is referred to as XS Store Sets.

The main optimization over original Store Sets is using multiple 'slots' for each LFST entry, thereby recording multiple dependent stores at once. This allows a load to track multiple dependencies with only one SSID. This is useful in situations where a load is dependent on different stores in different program contexts, or may have partial address overlap with multiple stores. There are further small optimizations to allocation policy which we omit here.

3 Method

This section presents the profile-guided memory dependence prediction (PG-MDP) technique. We put forward the concepts of store distances, distance thresholds, and how these are combined to find reliably memory independent loads. We then motivate the use of profiles to determine load labels, outlining the benefits provided over static analysis.

The PG-MDP workflow follows a standard profile-guided optimization pattern. Programs are first compiled with instrumentation then profiled using train inputs. During profiling, PG-MDP tracks reads and the most recent writes to memory addresses. Loads which have sufficiently long 'store distances' (detailed below) at least 95% of the time are determined to be sufficiently memory independent, and during profile-guided recompilation they are 'labelled' by being emitted as alternate opcodes, encoding an ISA hint without incurring additional instruction bandwidth. These new opcodes have the exact same semantics as the equivalent original load, but signal to the processor that they should skip querying the memory dependence predictor and issue as soon as possible. If they really do observe a dependency and trigger a memory order violation, this still causes a rollback as normal but no new entry is added in the MDP.

3.1 Store Distances & Labelling Thresholds

As overviewed in Section 2.1, when loads are issued in out-of-order execution they search the store queue (SQ) for older stores with overlapping addresses. Informally, a load's *store distance* refers to the number of prior SQ entries between the load and the dependent stores. A load depending on the most recent prior store has a store distance of 1. Formally, let x be the address of a load, and let the SQ contain store addresses $y_1, y_2, \ldots, y_n$ ordered from youngest (y_1) to oldest (y_n). The *store distance* is defined as the minimum index i such that $x = y_i$. If no such index exists, the store distance is ∞.

The profile-derived store distance for each load is found by recording every per-execution store distance on train inputs. Store distances that occur in 95% or more executions across all train inputs are selected as that load's profiled store distance. When multiple store distances are observed, with no single distance occurring in at least 95% of executions, the shortest observed distance is conservatively chosen.

Loads are then filtered by whether their profiled store distance is within a selected threshold. The premise of PG-MDP is that loads with an either infinite or sufficiently long store distances are good candidates to be labelled. The optimal store distance threshold for a specific target processor is found via binary search, choosing the threshold with the best average performance over train inputs. By using a representative selection of workloads to perform the search with, a single optimal store distance threshold can be found for the target processor rather than individual thresholds for each workload, meaning the search only has to be performed once.

The heuristic of filtering for long store distances captures four different types of behaviors:

- **(1) Memory independent loads:** Loads which read constant data (i.e. have infinite store distance).
- **(2) Rarely dependent loads:** Loads which observe short dependent store distances in <5% of executions.

- **(3) Structurally independent loads:** Loads with no dependent stores in-flight at the time the load is issued.
- **(4) Fast-to-Resolve Dependencies:** Loads which have in-flight yet resolved dependent stores, allowing forwarding.

Loads with behavior type (1) are always labelled regardless of the threshold value. Whether a load exhibits behaviors (2) to (4) depends on the target processor. For instance, a processor with a longer SQ will hold more in-flight stores at once, so only loads with longer store distances will be labelled. The proportion of loads that exhibit type (4) behavior is even further target specific, but is still correlated with a longer store distance as this puts more time between dispatching the store and issuing the load, making it more likely the store's address will be resolved when needed.

3.2 Why Profiles?

While profile-guided optimization (PGO) has long offered performance benefits, the technique has historically struggled to achieve adoption due to complicating the compilation workflow [5]. PGO also introduces concerns about the accuracy of generated profiles, and the potential to harm performance should real input differ too much from the train input.

Addressing adoption, prior work shows that use of PGO has risen significantly in recent years [13]. This can be seen in enterprise projects such as Firefox and Chrome web browsers, the Python interpreter and the GCC compiler. Performance-critical server workloads have also seen renewed attention to the benefits that PGO can provide [17]. This enforces the use of profiles as a reasonable approach to designing new software-hardware co-designs.

To test how PG-MDP generalizes, we compare which loads are labelled by profiles generated from both train and reference inputs for intspeed, shown in Figure 3. 'Positive' means the load is labelled and 'negative' means it is not, and so true positives are loads which are labelled in both profiles, false positives are loads only labelled in the train profile, and likewise for negatives. 'Missing' means the load does not appear in the train workload. These results suggest that profiles generated from train inputs do generalize effectively to reference inputs, with the majority of labelled loads as either true positives (TP) or true negatives (TN). Some missed potential is seen with false negatives (FN), but minimal harmful labels are introduced as false positives (FP). It is also important to note that in the case of load instructions which are unseen in the train inputs, these are never labelled and hence execute as normal. This allows PG-MDP to be more tolerant to workloads with high code coverage variation between inputs.

Lastly, profiles are able to provide further benefits than static analysis. Of the four types of load behavior referred to in Section 3.1, only two (types (1) and (3)) could theoretically be captured by perfect alias analysis. Type (2) would further require perfect memory dependence analysis, and there exists no analysis in conventional compiler design that attempts to capture the behavior in type (4). It is also important to note that the alias and dependence analysis offered by industrial compilers today is far from perfect [6]. Whereas stronger analyses do exist [2, 22], these are either only effective in domain-specific languages or do not scale to large programs, making them either less applicable or less feasible. In contrast, profiles allow much greater flexibility and coverage of load behavior, which pairs well with speculative execution where mistakes will not invalidate program semantics.

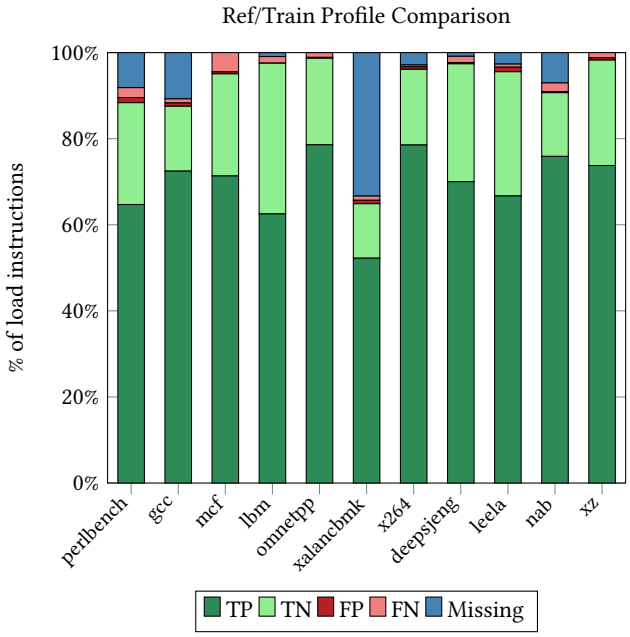

Figure 3: Comparison of labelled (static) loads between profiles generated from train and ref inputs with a store distance threshold of 13. Most labels are true positives/negatives when compared to the reference inputs, and very few are false positives.MB: Fig. 3 uses threshold 13, but Table 1 gives the small core threshold as 8 (13 is the medium core). If Fig. 3 is meant to support the small-core story, either use 8 or state why 13 is shown.

3.3 SK: To add once this section explains more in depth the mechanism

SK: Our mechanism requires only a binary ISA annotation that distinguishes conventional loads from labeled loads. In our RISC-V prototype, this annotation is realized through a new load instruction encoding. Other ISAs could provide the same semantic distinction using an available encoding, a reserved opcode, or an ISA-specific extension mechanism.

4 Evaluation

This section presents the experimental design and results of applying PG-MDP to SPECspeed 2017. We first demonstrate the extent to which the predictor working set can be reduced. We then present a breakdown of per-workload performance gains on the small core. Lastly we show how PG-MDP performs on a core scaled to double the size, i.e. the medium core.

4.1 Experimental Design

We use an optimized Gem5 fork [3] that includes an implementation of the XS Store Sets predictor to simulate a small and medium sized core. The full parameters of each model is detailed in Table 1.

We evaluate each AArch64-compiled SPECspeed 2017 workloads using up to 10 simpoints [20], with an interval of 100M instructions and an additional warm-up of 10M instruction. For ease of implementation, load labels are implemented in simulation using a text file of labelled load addresses. Gem5 is modified to bypass MDP lookups for labelled loads and to avoid creating new entries in the case of memory order violation. Violating labelled loads still roll back as normal and do not alter program semantics.

The modelled small core is intended to resemble out-of-order efficiency cores deployed today, with an instruction window length close to or matching those found in cores such as the Arm Cortex-A76 or Apple's M-series efficiency cores. We stress that this correspondence is only approximate, as Gem5 is a model with its own micro-architectural assumptions and features, and the design class is modelled with a configuration that is appropriate to Gem5 rather than by replicating any single processor. We configure XS Store Sets to use 128 SSIT entries and 64 LFST entries with two slots each, coming to a comparatively generous 300B of MDP storage for the baseline.

The medium core is used to explore how PG-MDP's impact changes with MDP size and available ILP, each of which are doubled over the small core. However, as previously discussed this use case is somewhat less realistic as larger cores are more likely to employ additional hardware techniques which influence the baseline predictor working set.

SK: We restrict our evaluation to Store Sets in the smaller cores because it represents the most relevant and representative memory dependence prediction approach for modern high-performance out-of-order processors. We do not consider other simple alternatives such as Store Vectors [**?**] because their primary contribution is an alternative predictor organization rather than improved dependence prediction accuracy. For large out-of-order cores, Store Vectors also incur a storage overhead that scales with the Store Queue size (e.g., 120 bits for a 120-entry Store Queue), in addition to tags and replacement metadata. Practical implementations further require logic to align the stored vector with the dynamic Store Queue organization through a barrel roll and to identify the youngest matching producer store per load, making this approach progressively less attractive as Store Queue capacity increases.

4.2 Performance

Figure 4 shows the percent IPC changes in each SPECspeed workload for the small core configuration. It can be seen that IPC gains have an uneven spread across workloads, with some seeing large benefits (up to 20%) and others next to none (or in the case of 654.*roms_s* a small regression, explained by Section **??**). This loosely correlates to the magnitude of false dependencies before and after applying PG-MDP seen in Figure 5. The largest gainers such as 625.*x*264_*s* and 600.*perlbench_s* all exhibit relatively higher false dependencies per kilo-instruction in the base case, and see this cut down significantly by over 80% with PG-MDP.

Listing 4.2 uses an example function from 625.*x*264_*s* to demonstrate how PG-MDP can be impactful. The code is a simplified version of *pixel_avg*, a hot function that causes a large portion of false dependencies. The loads on *src*1 and *src*2 never observe memory dependencies during program execution. As the loop is both short in instructions and part of a tight nest, these loads easily fall on the critical path during execution. Due to hash collisions, the loads are falsely made dependent on unrelated stores and the loop is unnecessarily serialized. Furthermore, the loop is compiled into three distinct variants that load different sizes depending on remaining pixels, increasing both static load and store count pressure and making hash collisions more likely. Using PG-MDP, loads from all iterations of all access sizes are free to issue in parallel, significantly improving ILP. Altogether this represents a perfect use case for PG-MDP and is a big contributor to the resulting performance gain.

Table 1: Parameters of processor components for each simulated model

		Small	Med.
Instruction Window	ROB:	128	256
	IQ:	77	154
	LQ:	41	85
	SQ:	26	66
Pipeline Width	Fetch/ Commit: Issue:	6 8	8 12
XS Store Sets	SSIT:	128	256
	LFST:	64	128
	Slots:	2	2
	Clear:	125k	125k
L1D	Size:	32KB	64KB
	MSHRs:	16	32
	Prefetcher:	Stride	
L1I	Size:	32KB	32KB
	MSHRs:	16	32
	Prefetcher:	Tagged	
L2	Size:	1MB	2MB
	MSHRs:	32	64
	Prefetcher:	Stride	
L3	Size:	2MB	4MB
	MSHRs:	64	64
	Prefetcher:	Stride	
Branch Predictor:		64KB TAGE-SC-L	
Store Distance Threshold:		8	22

```
void pixel_avg( uint8_t *dst, uint8_t *src1,
    uint8_t *src2,
              int i_width, int i_height )
{
    for ( int y = 0 .. i_height )
    {
        for( int x = 0 .. i_width )
            dst[x] = ( src1[x] + src2[x] + 1 ) >>
                1;
    }
}
```

In select cases PG-MDP can lead to a small performance regression due to false positives between the train and reference inputs, as discussed in Section 3.2. False positive labels are loads which behave memory independent on train inputs but exhibit dependencies on

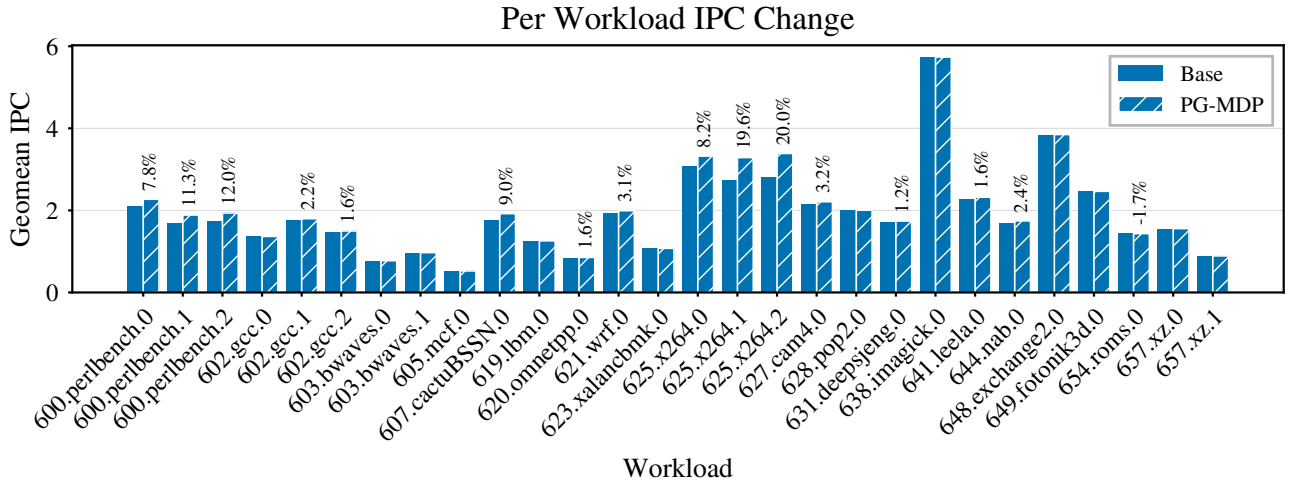

Figure 4: IPC % improvement with PG-MDP for each workload on the small core configuration. A handful of workloads benefit significantly, whereas others are largely unchanged. Annotations omit percent changes < 0.5.

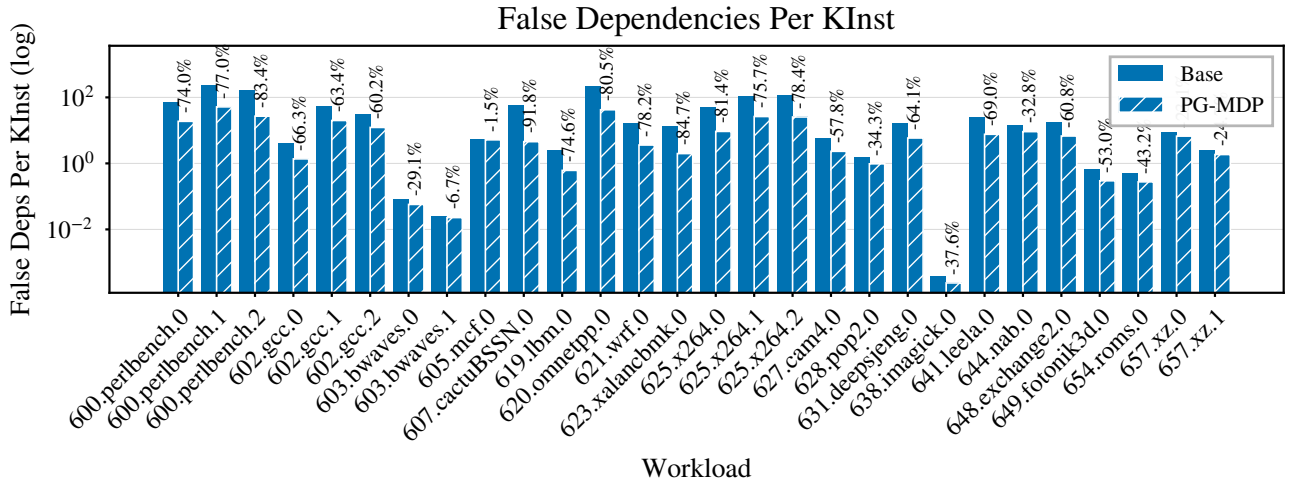

Figure 5: Percent change of false dependencies per kilo-instruction. PG-MDP benefits workloads most that by default have a higher than average rate of false dependencies, but is still able to cut false dependencies across all workloads.

ref inputs, causing an increase in memory order violations as seen in Figure 6. Although memory order violations are increased in all workloads, their impact is offset by the reduction in false dependencies. It is worth noting that violations are already rare events, measured in mega-instructions rather than kilo-instructions, so even a 100% increase can still remain in negligible ranges compared to the rate of false dependencies.

The degree to which violations increase is controlled by the threshold value discussed in Section 3.1. The optimal threshold may still provide a net performance gain while causing small regressions in specific workloads, and a larger threshold could instead be chosen to ensure regressions never occur. Violations could also be mitigated by specialising the PGO technique to each workload rather than the processor, which is discussed further in Section 7.

5 Related Work

Improving Memory Dependence Prediction with Static Analysis [16] tackles the same problem as this paper, but using static analysis to determine which load instructions to label. It demonstrates small IPC gains in select cases, but with an overall best-case geomean improvement of less than 0.1%. Furthermore, due to relying on static analysis it also sees notable performance regressions elsewhere, hurting viability. In comparison, by leveraging profiles we are able to achieve much higher IPC gains while also avoiding regressions. Our technique is also resistant to regressions on larger or more sophisticated MDP algorithms such as PHAST [11], which [16] is not.

Feedback-directed Memory Disambiguation Through Store Distance Analysis [9] proposes a similar profile-guided co-design to

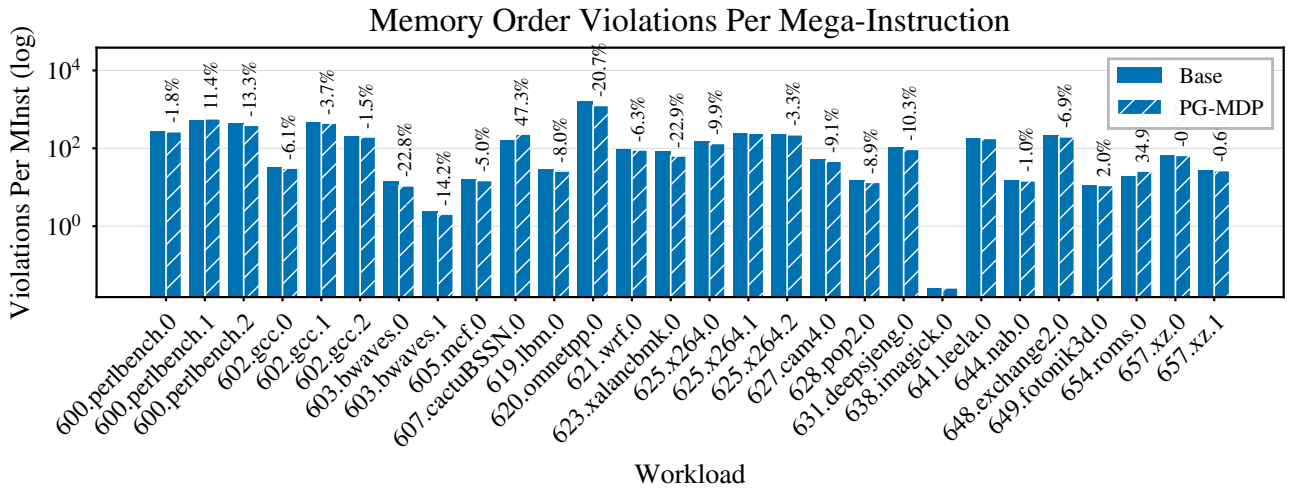

Figure 6: Percent change of memory order violations per Mega-instruction. Although relative % changes are high, absolute values remain low.

this paper, but with a more aggressive use case. In their work, the MDP is replaced altogether and load instructions are extended to include 4-bit distance fields indicating the store queue position they're likely to be dependent on. This achieves high accuracy compared to Store Sets, but comes at the cost of higher instruction bandwidth to encode predicted store distances, which is usually infeasible in modern processor designs where instruction bandwidth is critical. Larger processors would also likely require more bits to encode longer store queue distances. Furthermore, [9] could be more sensitive to profile accuracy than our work, as loads which do not appear in the profile must be assumed to be memory independent. In contrast, our technique is able to leave unseen loads to execute as normal.

Software-hardware Cooperative Memory Disambiguation [10] is a binary analysis co-design to reduce load queue pressure. Certain kinds of provably memory independent loads are labelled and prevented from being inserted into the load queue when issued. This technique is able to label 40% of static loads across SPEC2006 floatspeed (but dynamic percentage is unclear). This also incurs additional instruction bandwidth by inserting barrier-like instructions when entering loops to ensure labelled loads are only removed from the load queue when their parent loop reaches a steady state in the pipeline. LSQ entries must also store an additional bit to track whether these custom instructions are in-flight or not. This work presents an interesting proposal for reducing load queue pressure in area-constrained processors that lack space for hardware-based load elimination techniques, but has reduced viability due to currently lacking a mechanism for ensuring memory coherence in multi-core systems.

Constable: Improving Performance and Power Efficiency by Safely Eliminating Load Instruction Execution [4] is a pure hardware technique to track and eliminate stable loads which consistently read the same value from memory. This technique is able to eliminate both the memory read and address calculation altogether, greatly improving pipeline efficiency. It also demonstrates a wide coverage a variety of workloads, improving geomean IPC by 5.1%. For larger performance-oriented processors, this technique would most likely be more effective at improving the efficiency of load execution than our work, with a large overlap (albeit not total) of the types of load instructions targeted. However, at an estimated hardware overhead of 12.4KB, this technique may be infeasible for the area-constrained cores we target.

Other Memory Dependence Prediction Algorithms Besides Store Sets and PHAST [7, 11], other MDP algorithms include Store Vectors, MDP-TAGE, and MASCOT [15, 18, 19, 21] (MASCOT is an extension of PHAST, but with a focus on speculative-memory bypassing and performs similarly for strictly MDP). Each of these algorithms offer different trade-offs and advantages. We choose to evaluate XS Store Sets and PHAST because we understand them to be the optimal choices for small and large cores respectively, and that XS Store Sets provides, to the best of our knowledge, the closest representation of MDP algorithms found in real processors.

6 Threats to Validity

This section touches on possible threats to the validity of results presented in this paper.

6.1 Encoding Space for Load Labels

To communicate to the processor which loads PG-MDP selects, we have proposed introducing new opcodes in the ISA to label loads at zero area or bandwidth overhead. While more feasible for some ISAs such as RISCV, this may be difficult to implement in other pre-existing ISAs such as X86 due to limited available encoding space. Alternate proposals to this problem have been offered in [10]. Further options include introducing some level of overhead to apply our proposed technique, such as extending load opcodes to include a label bit, or reserving a bit in the register operand and adjusting register allocation accordingly.

6.2 Simulated Processor Models

The processor model we simulate specified in Table 1 does not reflect any real world processor configuration. Though we would like to use publicly known parameters from real processors (for instance from sources such as [1]), using Gem5 to model these processors can be dangerous, as the features which inform their optimal parameter values are likely absent in Gem5. As such we design the simulated models used in this paper by the intuition of what is reasonable for Gem5 specifically, while still within realistic area bounds for a real processor in our use case.

6.3 Target Specific Compilation

PG-MDP has been used in this paper assuming target specific compilation is possible in order to select the processor-specific optimal store distance threshold. In many cases though this is not the case. Enterprise software products are often compiled once and shipped to many different platforms. This would make short thresholds invalid, as they would introduce performance regressions in larger processors. A conservatively large threshold could be used for generic compilation, but this would likely also eliminate any performance gains in small processors too. One solution could be for processors known not to benefit from PG-MDP to ignore the ISA hint and execute labelled loads as normal.

6.4 Compilation Overhead

As a profiled method that instruments each load and store instruction, PG-MDP may incur additional overhead to the compilation workflow. In our own work we have not found the performance impact of instrumentation to be noticeable, but the memory footprint of tracking address accesses can be more significant, especially in large code footprint programs. In many industrial use cases this is still feasible, and prior work suggests there is tolerance to much more computationally expensive profile-guided techniques targeting general purpose workloads [25]. Nonetheless, there are various optimizations that could be applied to profiling memory accesses. For instance, statistical sampling could be used to only record every N access and provide an estimation for a load's store distance rather than a definitive value. PG-MDP could also be combined with Simpoints to only profile loads in hot program regions.

7 Future Work

Per-Workload Threshold Selection could achieve higher performance than per-core thresholds. As the threshold is untied to any architectural state, it is possible to select a different optimal store distance threshold for each individual workload. We chose not to do this for this work to prove that meaningful performance gains were possible with a generic threshold, and that the PG-MDP technique did not require a threshold search for each compiled program. However, there are use cases where this is feasible, and it would be worthwhile to evaluate the potential performance available for the additional compilation cost.

Just-in-Time Compilers (JITs) are a widely used run-time based compilation technique for dynamic languages. PG-MDP could pair well with these compilers for various reasons. Firstly, the additional workflow complexity of profiling can be automated by the run-time optimiser. Secondly, JITs make speculative optimizations about observed program behavior, and so could aggressively apply ISA labels to any loads which read from addresses not recently stored to. Lastly, dynamic languages tend to be implemented with many memory independent pointer-chasing loads. This suggests these workloads could have a high potential for performance gains from using PG-MDP.

Serverless Workloads are characterised as a collection of many short-lived workloads which only execute a few times, then not again for a long time with lots of unrelated code in-between. This effectively makes their execution always cold on the target processor, so high performance is difficult to achieve. When memory dependence predictors cannot learn dependencies fast enough to deliver performance gains (which is especially a concern for modern TAGE-like predictors such as PHAST), it may be preferable to disable memory dependence prediction entirely and conservatively stall loads on any unresolved stores. In this case, PG-MDP could potentially deliver substantial IPC gains, even on large cores, by allowing likely-independent loads to still issue immediately.

8 Conclusion

This paper proposed profile-guided memory dependence prediction (PG-MDP), a profile-guided co-design to label load instructions via their opcode and prevent them from making MDP queries. This was shown to reduce the predictor working set and allow a small XS Store Set predictor to match the performance of a large predictor at no area or instruction bandwidth cost. We showed PG-MDP effectively reduces false memory dependencies and delivers a geomean IPC gain of 1.47% in a simulated area-constrained core, which falls within 0.5% of a 16x larger predictor. We further showed that this technique does not introduce regressions as processor size scales, and can potentially offset the increased energy usage of modern MDP algorithms like PHAST in these processors.

References

[1] [n. d.]. Cardyaks Microarchitecture Cheat Sheet. https://docs.google.com/spreadsheets/d/18ln8SKIGRK5_6NymgdB9oLbTJCFwx0iFI-vUs6WFyuE/

[2] [n. d.]. MLIR Affine Dialect. https://mlir.llvm.org/docs/Dialects/Affine/.

[3] [n. d.]. Repo for the PHAST Gem5 Fork. https://gitlab.com/muke101/gem5-phast.

[4] Rahul Bera, Adithya Ranganathan, Joydeep Rakshit, Sujit Mahto, Anant V. Nori, Jayesh Gaur, Ataberk Olgun, Konstantinos Kanellopoulos, Mohammad Sadrosadati, Sreenivas Subramoney, and Onur Mutlu. 2025. Constable: Improving Performance and Power Efficiency by Safely Eliminating Load Instruction Execution. In *Proceedings of the 51st Annual International Symposium on Computer Architecture* (Buenos Aires, Argentina) *(ISCA '24)*. IEEE Press, 88–102. https://doi.org/10.1109/ISCA59077.2024.00017

[5] Dehao Chen, Neil Vachharajani, Robert Hundt, Shih-wei Liao, Vinodha Ramasamy, Paul Yuan, Wenguang Chen, and Weimin Zheng. 2010. Taming hardware event samples for FDO compilation. In *Proceedings of the 8th Annual IEEE/ACM International Symposium on Code Generation and Optimization* (Toronto, Ontario, Canada) *(CGO '10)*. Association for Computing Machinery, New York, NY, USA, 42–52. https://doi.org/10.1145/1772954.1772963

[6] Khushboo Chitre, Piyus Kedia, and Rahul Purandare. 2022. The Road Not Taken: Exploring Alias Analysis Based Optimizations Missed by the Compiler. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 153 (Oct. 2022), 25 pages. https://doi.org/10.1145/3563316

[7] George Z. Chrysos and Joel S. Emer. 1998. Memory Dependence Prediction Using Store Sets. In *Proceedings of the 25th Annual International Symposium on Computer Architecture, ISCA*. IEEE Computer Society, 142–153. https://doi.org/10.1109/ISCA.1998.694770

[8] Jason Lowe-Power et al. 2020. The gem5 Simulator: Version 20.0+. https://arxiv.org/abs/2007.03152. arXiv:2007.03152 [cs.AR]

[9] Changpeng Fang, Steve Carr, Soner Önder, and Zhenlin Wang. 2006. Feedback-Directed Memory Disambiguation through Store Distance Analysis. In *Proc. ICS '06* (Cairns, Queensland, Australia). 10 pages. https://doi.org/10.1145/1183401.1183440

[10] R. Huang, A. Garg, and M. Huang. 2006. Software-hardware cooperative memory disambiguation. In *Proc. HPCA, 2006.* 244–253. https://doi.org/10.1109/HPCA.2006.1598133

[11] Sebastian S. Kim and Alberto Ros. 2024. Effective Context-Sensitive Memory Dependence Prediction. In *30th Symposium on High Performance Computer Architecture (HPCA).* IEEE Computer Society, Edinburgh, Scotland.

[12] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: an integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture* (New York, New York) *(MICRO 42).* Association for Computing Machinery, New York, NY, USA, 469–480. https://doi.org/10.1145/1669112.1669172

[13] Bingxin Liu, Yinghui Huang, Jianhua Gao, Jianjun Shi, Yongpeng Liu, Yipin Sun, and Weixing Ji. 2025. From Profiling to Optimization: Unveiling the Profile Guided Optimization. arXiv:2507.16649 [cs.PF] https://arxiv.org/abs/2507.16649

[14] Lasse Natvig Marius Grannaes, Magnus Jahre. 2011. Storage Efficient Hardware Prefetching using Delta-Correlating Predicting Tables. *Journal of Instruction-Level Parallelism.* https://jilp.org/dpc/online/papers/02grannaes.pdf

[15] Karl H. Mose, Sebastian S. Kim, Alberto Ros, Timothy M. Jones, and Robert D. Mullins. 2025. Mascot: Predicting Memory Dependencies and Opportunities for Speculative Memory Bypassing. In *31st Symposium on High Performance Computer Architecture (HPCA).* IEEE Computer Society, Las Vegas, NV, USA, 59–71.

[16] Luke Panayi, Rohan Gandhi, Jim Whittaker, Vassilios Chouliaras, Martin Berger, and Paul Kelly. 2024. Improving Memory Dependence Prediction with Static Analysis. In *Architecture of Computing Systems*, Dietmar Fey, Benno Stabernack, Stefan Lankes, Mathias Pacher, and Thilo Pionteck (Eds.). Springer Nature Switzerland, Cham, 301–315.

[17] Maksim Panchenko, Rafael Auler, Bill Nell, and Guilherme Ottoni. 2018. BOLT: A Practical Binary Optimizer for Data Centers and Beyond. arXiv:1807.06735 [cs.PL] https://arxiv.org/abs/1807.06735

[18] Arthur Perais and André Seznec. 2017. Storage-Free Memory Dependency Prediction. *IEEE Comput. Archit. Lett.* 16, 2 (Nov. 2017), 149–152. https://doi.org/10.1109/LCA.2016.2628379

[19] Arthur Perais and André Seznec. 2018. Cost effective speculation with the omnipredictor. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques, PACT.* ACM, 25:1–25:13. https://doi.org/10.1145/3243176.3243208

[20] Erez Perelman, Greg Hamerly, Michael Van Biesbrouck, Timothy Sherwood, and Brad Calder. 2003. Using SimPoint for Accurate and Efficient Simulation. *SIGMETRICS Perform. Eval. Rev.* 31, 1 (Jun 2003), 318–319. https://doi.org/10.1145/885651.781076

[21] Samantika Subramaniam and Gabriel H. Loh. 2006. Store vectors for scalable memory dependence prediction and scheduling. In *12th International Symposium on High-Performance Computer Architecture, HPCA.* IEEE Computer Society, 65–76. https://doi.org/10.1109/HPCA.2006.1598113

[22] Yulei Sui and Jingling Xue. 2016. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th International Conference on Compiler Construction.* ACM, 265–266.

[23] XiangShan Team. 2026. XiangShan GEM5 Github Repo. https://github.com/OpenXiangShan/GEM5.

[24] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, Jiawei Lin, Tong Liu, Zhigang Liu, Jiazhan Tan, Huaqiang Wang, Huizhe Wang, Kaifan Wang, Chuanqi Zhang, Fawang Zhang, Linjuan Zhang, Zifei Zhang, Yangyang Zhao, Yaoyang Zhou, Yike Zhou, Jiangrui Zou, Ye Cai, Dandan Huan, Zusong Li, Jiye Zhao, Zihao Chen, Wei He, Qiyuan Quan, Xingwu Liu, Sa Wang, Kan Shi, Ninghui Sun, and Yungang Bao. 2022. Towards Developing High Performance RISC-V Processors Using Agile Methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO).* 1178–1199. https://doi.org/10.1109/MICRO56248.2022.00080

[25] Siavash Zangeneh, Stephen Pruett, Sangkug Lym, and Yale N. Patt. 2020. BranchNet: A Convolutional Neural Network to Predict Hard-To-Predict Branches. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO).* 118–130. https://doi.org/10.1109/MICRO50266.2020.00022